

(3) 洛谷 P3195(玩具装箱): DP 状态转移方程为 $dp[i] = \min\{dp[j] + (\text{sum}[i] + i - \text{sum}[j] - j - L - 1)^2\}$ 。

(4) 洛谷 P4072(征途): 二维斜率优化, DP 状态转移方程为 $dp[i][p] = \min\{dp[j][p-1] + (s[i] - s[j])^2\}$ 。

5.10

四边形不等式优化



视频讲解

将四边形不等式(Quadrangle Inequality)应用于 DP 优化,起源于 1971 年 Knuth 的一篇论文^①,用来解决最优二叉搜索树问题。1980 年,储枫(F. Frances Yao, 姚期智的夫人)做了深入研究^②,扩展为一般性的 DP 优化方法,把一些 DP 问题复杂度从 $O(n^3)$ 优化到 $O(n^2)$ 。所以,这个方法又称为 Knuth-Yao DP Speedup Theorem。

四边形不等式应用于 DP 优化的证明比较复杂,如果先给出定义和证明,会让人迷惑,所以本节按这样的顺序讲解:先用应用场合引导出四边形不等式的概念,再进行定义和证明,最后用例题巩固。四边形不等式优化虽然理论复杂,但是编码很简单。

5.10.1 应用场合

有一些常见的 DP 问题,通常是区间 DP 问题,它的状态转移方程为

$$dp[i][j] = \min\{dp[i][k] + dp[k+1][j] + w[i][j]\}$$

其中, $i \leq k < j$, 初始值 $dp[i][i]$ 已知。 $\min()$ 也可以是 $\max()$ 。

方程的含义如下。

(1) $dp[i][j]$ 表示从状态 i 到状态 j 的最小花费。题目一般是求 $dp[1][n]$, 即从起点 1 到终点 n 的最小花费。

(2) $dp[i][k] + dp[k+1][j]$ 体现了递推关系。 k 在 i 和 j 之间滑动, k 有一个最优值, 使 $dp[i][j]$ 最小。

(3) $w[i][j]$ 的性质非常重要。 $w[i][j]$ 是与题目有关的费用, 如果它满足四边形不等式和单调性, 那么计算时就能进行四边形不等式优化。

这类问题的经典的例子是“石子合并”^③, 在 5.5 节有介绍, 它的状态定义就是上面的 $dp[i][j]$, 表示区间 $[i, j]$ 上的最优值, $w[i][j]$ 是从第 i 堆石子合并到第 j 堆石子的总数量。

在阅读后面的讲解时, 读者可以对照“石子合并”这个例子来理解。注意, 石子合并有多种情况和解法, 详情见本节例题洛谷 P1880。

重新回顾 5.5 节求 $dp[i][j]$ 的代码, 作为后面优化代码的对照。代码中有三重循环, 复杂度为 $O(n^3)$ 。

① Knuth D E. Optimum Binary Search Trees[J]. Acta Informatica, 1971, 1: 14-25.

② Yao F F. Efficient Dynamic Programming Using Quadrangle Inequalities[C]//Proceedings of the 12th Annual ACM Symposium on Theory of Computing, 1980. http://www.cs.ust.hk/mjg_lib/bibs/DPSu/DPSu_Files/p429-yao.pdf.

③ 参考《算法竞赛入门到进阶》7.3 节“区间 DP”中的石子合并问题。


```

1  for(int i = 1; i <= n; i++) dp[i][i] = 0;           //初始值
2  for(int len = 2; len <= n; len++)                //len 为区间[i,j]的长度.从小区间扩展到大区间
3      for(int i = 1; i <= n - len + 1; i++){        // 区间起点 i
4          int j = i + len - 1;                      // 区间终点 j, i <= j <= n
5          for(int k = i; k < j; k++)                //大区间[i,j]从小区间[i,k]和[k+1,j]转移而来
6              dp[i][j] = min(dp[i][j], dp[i][k] + dp[k+1][j] + w[i][j]);
7  }

```

5.10.2 四边形不等式优化操作

只需一个简单的优化操作,就能把上面代码的时间复杂度降为 $O(n^2)$ 。这个操作就是把循环 $i \leq k < j$ 改为 $s[i][j-1] \leq k \leq s[i+1][j]$ 。 $s[i][j]$ 记录 $i \sim j$ 的最优分割点,在计算 $dp[i][j]$ 的最小值时得到区间 $[i, j]$ 的分割点 k ,记录在 $s[i][j]$ 中,用于下一次循环。

这个优化称为四边形不等式优化。下面给出优化后的代码,优化见加粗部分。

```

1  for(i = 1; i <= n; i++){
2      dp[i][i] = 0;
3      s[i][i] = i;                                //s[i][i]的初始值
4  }
5  for(int len = 2; len <= n; len++){
6      for(int i = 1; i <= n - len + 1; i++){
7          int j = i + len - 1;
8          for(k = s[i][j-1]; k <= s[i+1][j]; k++){ //缩小循环范围
9              if(dp[i][j] > dp[i][k] + dp[k+1][j] + w[i][j]){ //是否更优
10                 dp[i][j] = dp[i][k] + dp[k+1][j] + w[i][j];
11                 s[i][j] = k;                               //更新最佳分割点
12             }
13         }
14     }

```

代码的复杂度如何?

代码中 i 和 k 这两个循环,优化前复杂度为 $O(n^2)$ 。优化后,每个 i 内部的 k 的循环次数为 $s[i+1][j] - s[i][j-1]$,其中 $j = i + \text{len} - 1$ 。那么:

$i=1$ 时, k 循环 $s[2][\text{len}] - s[1][\text{len}-1]$ 次;

$i=2$ 时, k 循环 $s[3][\text{len}+1] - s[2][\text{len}]$ 次;

...

$i=n-\text{len}+1$ 时, k 循环 $s[n-\text{len}+2][n] - s[n-\text{len}+1][n+1]$ 次。

将上述次数相加,总次数为

$$s[2][\text{len}] - s[1, \text{len} - 1] + s[3][\text{len} + 1] - s[2, \text{len}] + \cdots + s[n + 1, n] - s[n][n] \\ = s[n - \text{len} + 2][n] - s[1][\text{len} - 1] < n$$

i 和 k 循环的时间复杂度优化到了 $O(n)$,总复杂度从 $O(n^3)$ 优化到了 $O(n^2)$ 。

在后面的四边形不等式定理证明中,将更严谨地证明复杂度。

图 5.25 所示为四边形不等式优化效果, s_1 是区间 $[i, j-1]$ 的最优分割点, s_2 是区间 $[i+1, j]$ 的最优分割点。

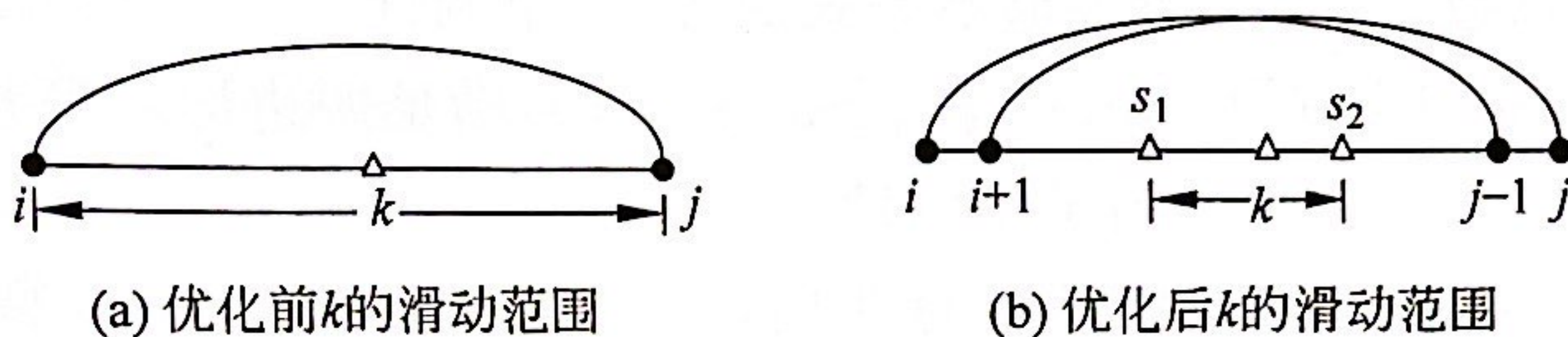


图 5.25 四边形不等式优化效果

读者对代码可能有两个疑问：

- (1) 为什么能够把 $i \leq k < j$ 缩小到 $s[i][j-1] \leq k \leq s[i+1][j]$?
- (2) $s[i][j-1] \leq s[i+1][j]$ 成立吗?

下面给出四边形不等式优化的正确性和复杂度的严谨证明, 会解答这两个问题。

5.10.3 四边形不等式定义和单调性定义